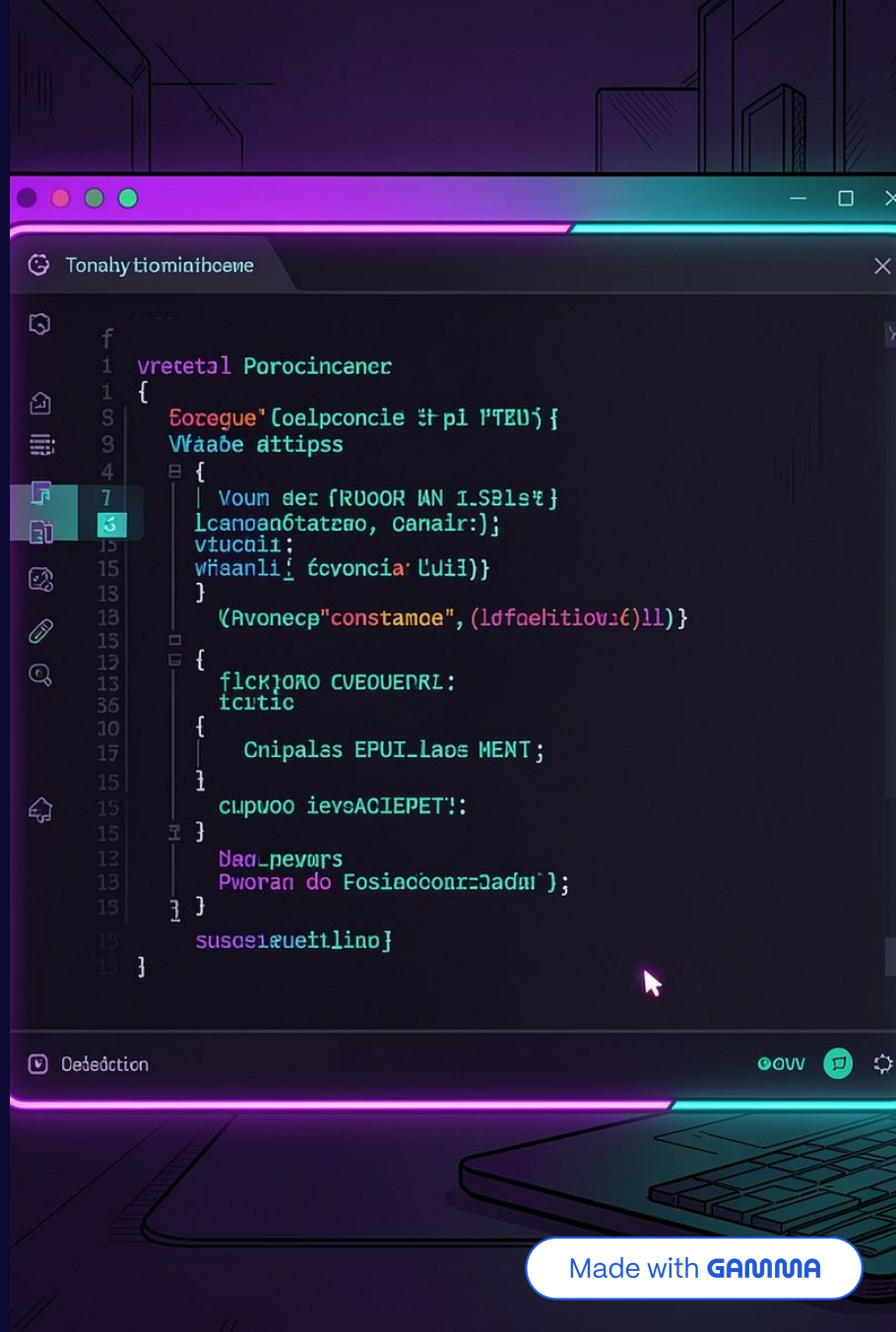
# Container Shell Management

*A Modular Linux Shell for Command, Process and Signal Management*

---

**Operating Systems & Systems Programming** · Section A7 · Team 15

- Vidya Keerthi Chitturi — 2510030372
- Badam Hasika — 2510030403
- Shaik Asha Parveen — 2510030225
- Madi Amulya — 2510030158



Made with GAMMA

# Project Overview

Container Shell Management is a custom command-line shell in C that implements core Linux OS concepts: command parsing, process creation, execution, synchronization, built-in commands, environment variables, and signal handling.

## Objectives

01

Build a functional Linux shell

02

Parse user commands

03

Execute external Linux programs

04

Implement shell built-in commands

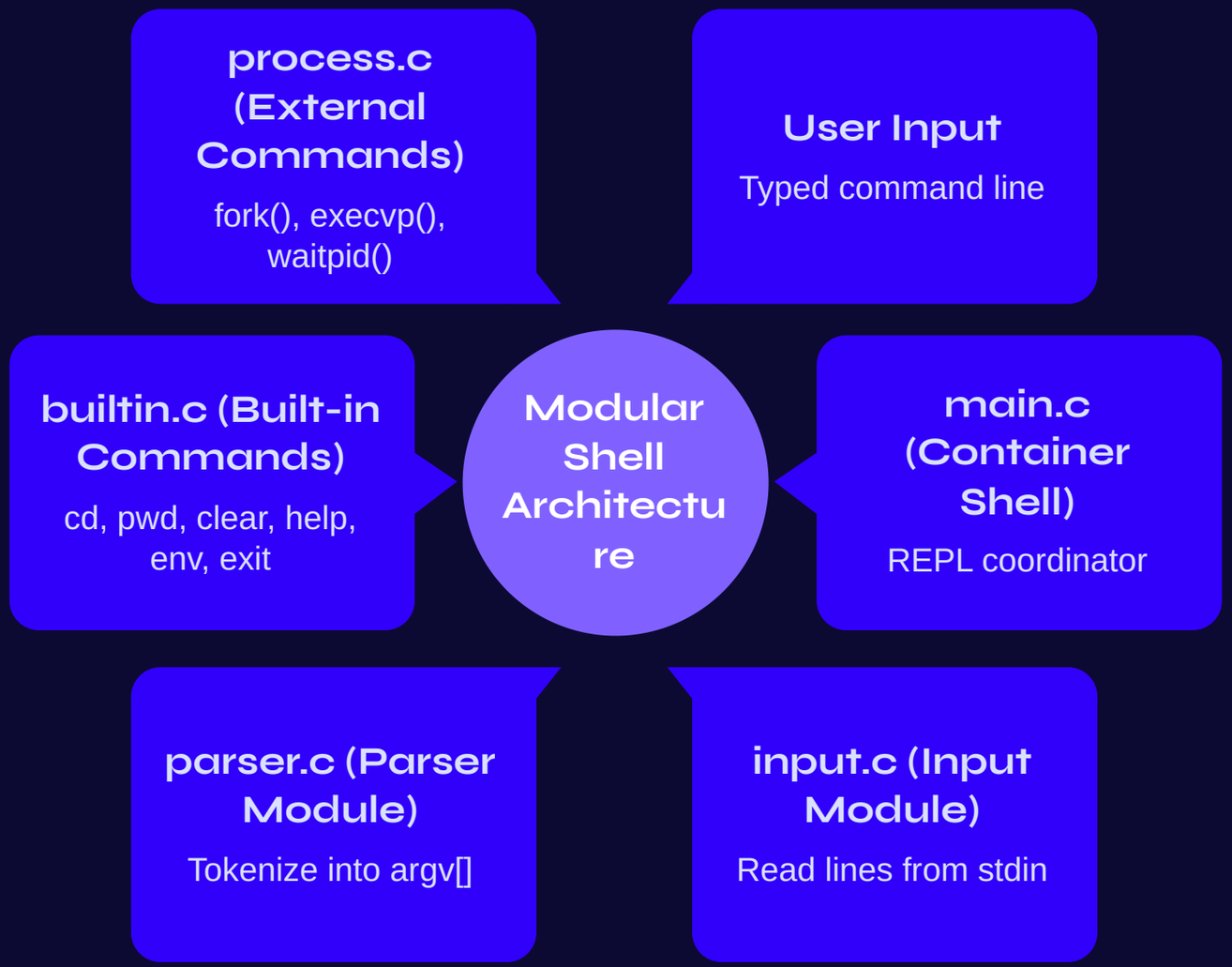
05

Manage processes and signals safely

Made with GAMMA

ARCHITECTURE

# Modular System Architecture

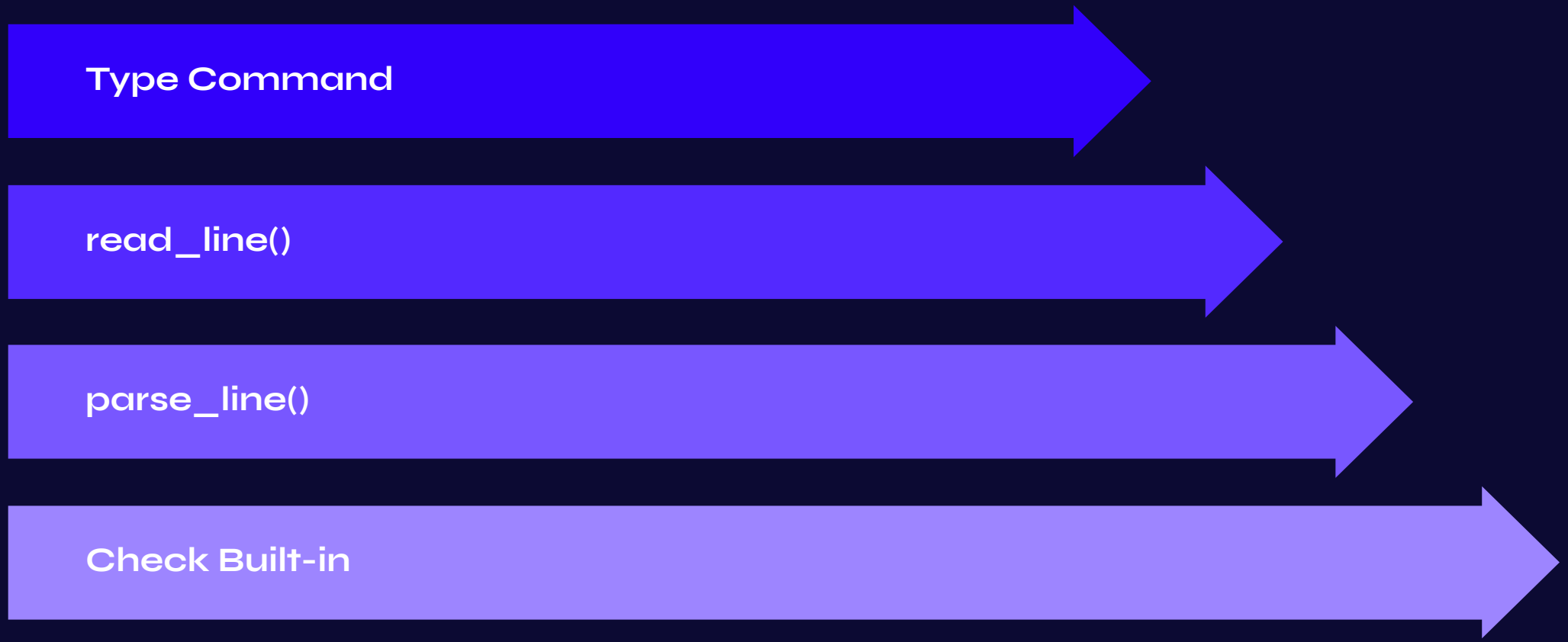



Each module has a single responsibility. `main.c` coordinates the REPL loop; `input.c` and `parser.c` handle user input; `builtin.c` and `process.c` execute commands; `signals.c` manages asynchronous notifications.

Made with GAMMA

COMMAND PROCESSING

# From User Input to Command Execution




`strtok()` splits input into whitespace-separated tokens. For `ls -l`: `argv[0]="ls"`, `argv[1]="-l"`, `argv[2]=NULL`. Built-ins execute in the shell process; external commands spawn a child via `fork()`.

Made with GAMMA



PROCESS MANAGEMENT

# Process Creation & Synchronization

### fork()

Duplicates the parent shell into a child process with an identical memory image.

### execvp()

Replaces the child's image with the requested external program, searching `PATH`.

### waitpid()

Parent blocks until the child exits, preventing orphaned processes.

### perror()

Prints a descriptive error when `fork()`, `execvp()`, or `waitpid()` fails.

**OS Concept:** A **program** is passive code on disk; a **process** is its active, running instance.

Made with GAMMA

BUILT-INS & ENVIRONMENT

# Shell Built-ins and Environment Variables

## Built-in Commands

| Command | Function |
| --- | --- |
| `cd` | Change directory |
| `pwd` | Print working directory |
| `clear` | Clear terminal |
| `help` | List available commands |
| `env` | Show environment variables |
| `exit` | Terminate shell |

`cd` → `chdir()` · `pwd` → `getcwd()` · `env` → `getenv()`

### Why is `cd` a Built-in?

`cd` must run *inside* the shell process. A child process cannot change its parent's working directory.

Environment variables `HOME`, `USER`, and `PATH` are accessed via `getenv()`.

Made with GAMMA

SIGNAL HANDLING

# Signals and Zombie-Process Cleanup

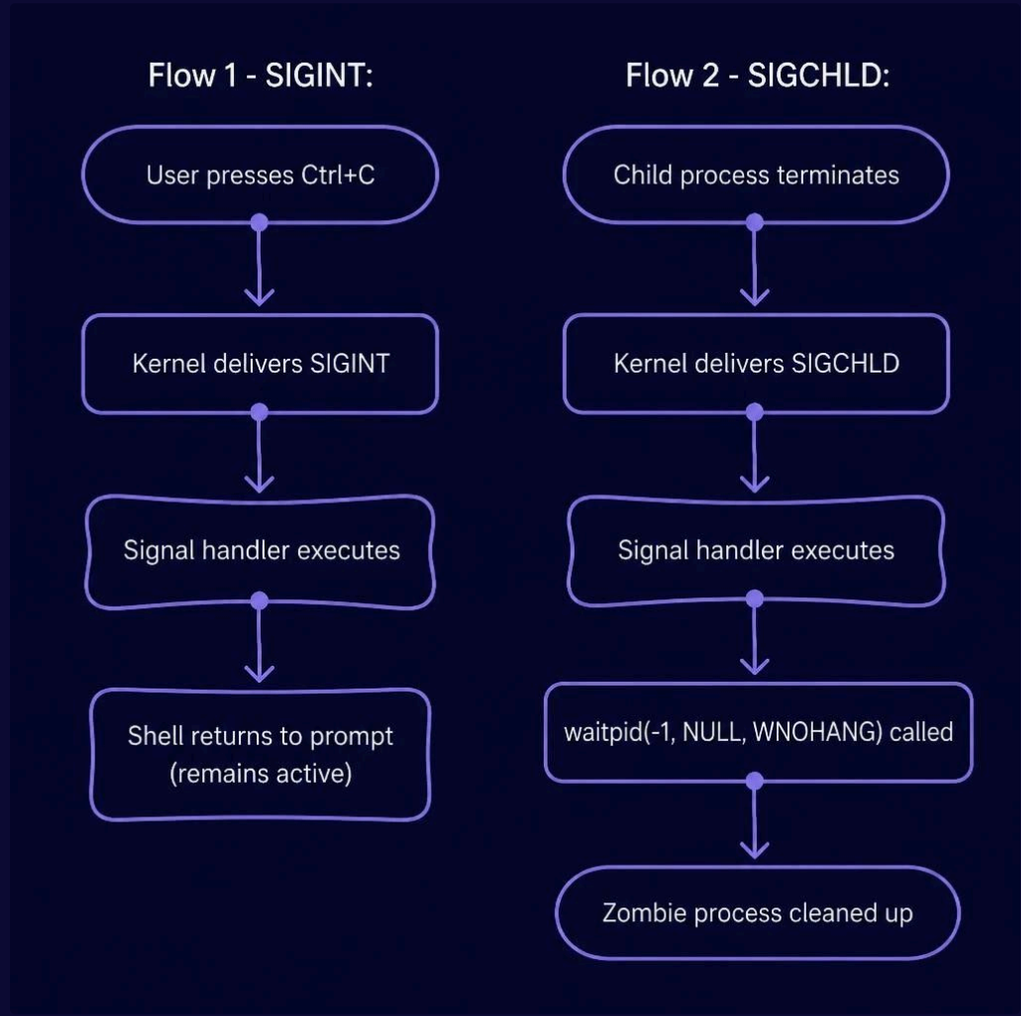
## SIGINT — Ctrl+C

When the user presses **Ctrl+C**, the kernel delivers `SIGINT` to the shell. The handler returns control to the prompt — the shell does not terminate.

## SIGCHLD — Child Exit

When a child process terminates, the kernel sends `SIGCHLD`. The handler calls `waitpid(-1, NULL, WNOHANG)` to reclaim the child and prevent a zombie.

> `WNOHANG` makes the call non-blocking — it returns immediately if no child has exited.




Made with GAMMA

DEVELOPMENT JOURNEY

# Six-Week Development Journey

1

## Week 1–2

Shell foundation: REPL loop, Makefile, dynamic input with `malloc()`/`realloc()`, memory cleanup with `free()`, `exit` command.

2

## Week 3

Command parsing: `strtok()` tokenization, `argv[]` construction, separation of command and arguments.

3

## Week 4

Process execution: `fork()`, `execvp()`, `waitpid()` synchronization, error reporting with `perror()`.

4

## Week 5

Built-in commands: `cd`, `pwd`, `clear`, `help`, `env`, `exit`. Environment variables: `HOME`, `USER`, `PATH`.

5

## Week 6

Signal handling: `SIGINT` for Ctrl+C, `SIGCHLD` for child exit, zombie cleanup with `waitpid(-1, NULL, WNOHANG)`. Separate `signals.c`/`signals.h` module.

**Each week extended the previous shell implementation into a more complete process-management system.**

Made with GAMMA

OUTCOME

# Final Outcome

## Completed System

- Dynamic Input
- Command Parsing
- External Commands
- Built-in Commands
- Environment Variables
- Process Management
- Signal Handling

## Key Takeaways

- Built a functional custom Linux shell in C.
- Applied `fork()`, `execvp()`, and `waitpid()` in practice.
- Added built-in commands and environment-variable support.
- Implemented signal handling and zombie-process cleanup.

This project demonstrates how a shell interacts with Linux processes, system calls, environment variables, and signals.

---

# Thank You

Made with GAMMA